Kapitel 3

Relationale Algebra

Anfragesprachen

- **Anfragesprache (QL)** ermöglicht es Benutzern, Daten aus einer Datenbank abzurufen und zu manipulieren
- Relationales Modell unterstützt einfache, aber leistungsfähige Anfragesprachen
  - Starke formale Grundlage basierend auf Logik
  - Ermöglicht Optimierung von Anfragen

Anfragesprache $\neq$ Programmiersprache

- Bieten einfachen Zugriff auf große Datenmengen
- Nicht für komplexe Anwendungen oder Berechnungen gedacht
- Anfragesprachen müssen nicht Turing-vollständig sein

Kategorien von Anfragesprachen

- **Deklarativ:** Fokus auf „*was* abgerufen werden soll“
- **Prozedural:** Fokus auf „*wie* auf die Daten zugegriffen wird“

Anfragesprachen (Fortsetzung)

- Zwei mathematische Anfragesprachen bilden die Grundlage moderner Anfragesprachen (z. B. SQL) und deren Implementierung
  - **Relationale Algebra:** Prozedurale Sprache, besonders nützlich zur Darstellung von Anfrageausführungsplänen und Optimierungstechniken ($\Rightarrow$ SQL)
  - **Relationaler Kalkül:** Deklarative, logikbasierte Sprache ($\Rightarrow$ QBE, Query-By-Example)

Das Verständnis von Algebra und Kalkül ist der Schlüssel zum Verständnis von SQL, Anfrageverarbeitung und -optimierung

Einführung: Relationale Algebra

Motivation und Überblick

- **Relationale Algebra:** formale Sprache zur Beschreibung von Datenbankanfragen im relationalen Modell
- Theoretische Grundlage für relationale Anfragesprachen wie SQL
- Alle Operationen der relationalen Algebra sind auf Mengen von Tupeln (Relationen) definiert

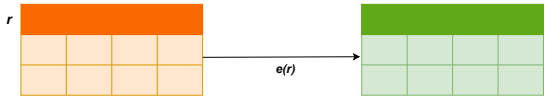
Warum relationale Algebra?

- Formale, mathematisch saubere Beschreibung von Anfragen
- Grundlage für die interne Repräsentation und Optimierung von SQL-Anfragen (z. B. Ausführungspläne)
- Verständliches Modell für Lehre, Analyse und Implementierung
- Beschränkt, aber dennoch ausdrucksstark – ideal zur systematischen Anfrageverarbeitung

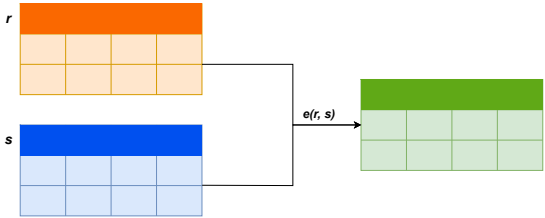
Relationale Algebra beschreibt „wie“ Daten verarbeitet werden — sie ist prozedural

Relationale Algebra

- Berechnungen auf Relationen bestehen aus
  - Operanden: Relationen
  - Operatoren: Ausdrücke der relationalen Algebra
- Operatoren nehmen eine (unäre) oder zwei (binäre) Relationen als Eingabe
- Ergebnis einer Operation ist immer eine (neue!) Relation
- Relationale Algebra ist „abgeschlossen“ über Relationen



Unäre Operation der relationalen Algebra



Binäre Operation der relationalen Algebra

Relationale Algebra

Grundoperationen

Unäre Operationen

- **Selektion:** Wählt eine Teilmenge von Tupeln aus einer Relation aus
- **Projektion:** Entfernt unerwünschte Spalten aus einer Relation
- **Umbenennung:** Benennt Attribut(e) und Relation(en) um

Binäre Operationen

- **Kreuzprodukt:** Kombiniert Tupel aus zwei Relationen
- **Vereinigung:** Tupel aus Relation 1 und Relation 2
- **Differenz:** Tupel aus Relation 1, die nicht in Relation 2 vorkommen

Da die relationale Algebra über Relationen abgeschlossen ist, können komplexere Anfragen durch Kombination mehrerer Operatoren formuliert werden (→ Minimalität)

Arithmetische Ausdrücke

Ein arithmetischer Ausdruck in relationaler Algebra kann rekursiv definiert werden:

- Basisfall: jede Konstante und jeder Attributausdruck sind ein arithmetischer Ausdruck
- Wenn E_1 und E_2 arithmetische Ausdrücke sind, dann auch
  - $E_1 + E_2$
  - $E_1 - E_2$
  - $E_1 \cdot E_2$
  - $E_1 \div E_2$
- Arithmetische Ausdrücke können umbenannt werden

Arithmetische Ausdrücke

Wo können arithmetische Ausdrücke auftreten?

- Projektion
- Selektion
- Verbund (Join)
- Aggregation
- Umbenennung

aber nicht in

- Mengenoperationen
- Kartesisches Produkt

Selektion

Wähle alle Tupel aus einer Relation aus, für die eine aussagenlogische Formel P wahr ist

Notation

$$\sigma_p(r)$$

- r : Relation (Name)
- P : Formel der Aussagenlogik, bestehend aus Bedingungen der Form:

```
<Attribut> = <Attribut> oder <Konstante>
```

Selektion

Wähle alle Tupel aus einer Relation aus, für die eine aussagenlogische Formel P wahr ist

Notation

$$\sigma_p(r)$$

- r : Relation (Name)
- P : Formel der Aussagenlogik, bestehend aus Bedingungen der Form:

```
<Attribut> = <Attribut> oder <Konstante>
```

Definition

$$\sigma_p(r) := \{ t \mid t \in r \text{ und } P(t) = \texttt{true} \}$$

- Vergleichsoperatoren: $=, \neq, <, >, \geqslant, \leqslant$
- Bedingungen können mit logischen Operatoren kombiniert werden: $\wedge$ (und), $\vee$ (oder), $\neg$ (nicht)

Selektion

Beispiel

r

A	B	C
1	1	7
2	2	3
3	3	6
4	5	9

$$\sigma_{(A=B)\wedge(C>5)}(r) \Longrightarrow$$

A	B	C
1	1	7
3	3	6

r

A	B	C
1	1	7
2	2	3
3	3	6
4	5	9

$$\sigma_{(A\cdot B>C)}(r) \Longrightarrow$$

A	B	C
2	2	3
3	3	6
4	5	9

Projektion

Gibt eine neue Relation mit nur den ausgewählten Attributen zurück

Notation

$$\pi_{A_1,\ldots,A_k}(r)$$

- $A_1, \ldots, A_k$: Attribute
- r : Relation

Hinweise

- Auch die Attributreihenfolge kann sich ändern
- Kardinalität der Ergebnisrelation s : $\mid \pi_{A_1,\ldots,A_k}(r) \mid \leqslant \mid r \mid$

Projektion

Gibt eine neue Relation mit nur den ausgewählten Attributen zurück

Notation

$$\pi_{A_1,\ldots,A_k}(r)$$

- $A_1, \ldots, A_k$: Attribute
- r : Relation

Hinweise

- Auch die Attributreihenfolge kann sich ändern
- Kardinalität der Ergebnisrelation s : $\mid \pi_{A_1,\ldots,A_k}(r) \mid \leqslant \mid r \mid$

  $\Rightarrow$ Doppelte Zeilen, die durch die Projektion entstehen könnten, werden aus dem Ergebnis entfernt

Projektion

Beispiel

r

A	**B**	**C**
1	2	3
1	3	3
2	3	4

$$\pi_{A,C}(r) \Longrightarrow$$

A	**C**
1	3
2	4

r

A	**B**	**C**
1	2	3
1	3	3
2	3	4

$$\pi_{2 \cdot A}(r) \Longrightarrow$$

2 · A
2
4

Vereinigung

Kombiniert zwei Relationen, indem Tupel aus beiden in eine neue Ergebnisrelation übernommen werden

Notation

$$r \cup s$$

(r, s Relationen)

Definition

$$r \cup s := \{ t \mid t \in r \vee t \in s \}$$

Voraussetzungen

- Relationen r und s müssen dieselbe Stelligkeit (Arity) besitzen
- Attributdomänen müssen kompatibel sein (z. B. gleicher Datentyp in gleicher Spalte)
- Es gilt (wegen Duplikateliminierung): $| r \cup s | \leqslant | r | + | s |$

Vereinigung

Beispiel

Gegeben seien die Relationen r und s

r

A	B
11	1
11	2
12	1

s

A	B
11	2
12	3

$r \cup s \Rightarrow$

A	B
11	1
11	2
12	1
12	3

Differenz

Liefert nur die Tupel zurück, die in einer Relation, aber nicht in der anderen vorkommen

Notation

$$r - s$$

(r, s Relationen)

Definition

$$r - s := \{ t \mid t \in r \wedge t \notin s \}$$

Voraussetzungen

- Relationen r und s müssen dieselbe Stelligkeit besitzen
- Attributdomänen müssen kompatibel sein

Hinweise

- Kardinalität: $| r - s | = | r | - | r \cap s |$
- Grenzen: $0 \leqslant | r - s | \leqslant | r |$

Differenz

Beispiel

Gegeben seien die Relationen r und s

r

A	B
11	1
11	2
12	1

s

A	B
11	2
12	3

$r - s$ $\Longrightarrow$

A	B
11	1
12	1

Kartesisches Produkt

Erzeugt eine neue Relation durch Kombination aller möglichen Tupelpaare beider Eingaberelationen

Notation

$$r \times s$$

(r, s Relationen)

Definition

$$\{ (a_1, ..., a_m, b_1, ..., b_n) \mid (a_1, ..., a_m) \in r \wedge (b_1, ..., b_n) \in s \}$$

wobei $a_1, ..., a_m$ Attribute von r und $b_1, ..., b_n$ Attribute von s sind

Hinweise

- Anzahl der Tupel: $| r \times s | = | r | \cdot | s |$

Kartesisches Produkt

Beispiel

Mitarbeiter

PersNr	Name	Vorname	Abteilung
001	Huber	Erwin	01
002	Mayer	Hugo	01
003	Müller	Anton	02

Abteilung

AbtNr	AbtName
01	Buchhaltung
02	Produktion

Mitarbeiter $\times$ *Abteilung*

PersNr	Name	Vorname	Abteilung	AbtNr	AbtName
001	Huber	Erwin	01	01	Buchhaltung
001	Huber	Erwin	01	02	Produktion
002	Mayer	Hugo	01	01	Buchhaltung
002	Mayer	Hugo	01	02	Produktion
003	Müller	Anton	02	01	Buchhaltung
003	Müller	Anton	02	02	Produktion

Umbenennung

- Ermöglicht es, Ergebnissen von Ausdrücken der relationalen Algebra Namen zu geben (zur Referenzierung)
- Erlaubt es, eine Relation mehrfach mit verschiedenen Namen zu verwenden (z.B. bei Selbstvergleichen)
- Beispiel:
  - $\rho_x(E)$ bezeichnet den Ausdruck E unter dem neuen Namen x
- Wenn der Ausdruck E die Stelligkeit k hat, dann bezeichnet $\rho_{x(A_1,A_2,\ldots,A_k)}(E)$ den Ausdruck E mit neuem Relationen- und neuen Attributnamen

Zusätzliche Operatoren

- Erhöhen nicht die Ausdrucksstärke der relationalen Algebra, vereinfachen jedoch viele komplexe Anfragen:
  - Durchschnitt (Schnittmenge) ($\cap$)
  - Division ($\div$)
  - Natürlicher Verbund ($\bowtie$)
  - Bedingter Verbund / Theta-Verbund ($\bowtie_c$)

Schnittmenge

Für zwei Relationen werden nur die Tupel zurückgegeben, die in beiden Relationen vorkommen

Notation

$$r \cap s$$

(r, s Relationen)

Definition

$$r \cap s := \{ t \mid t \in r \wedge t \in s \}$$

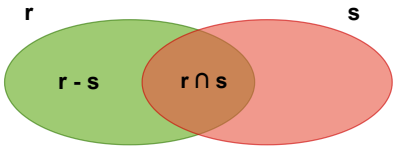
Voraussetzungen

- Relationen r und s müssen dieselbe Stelligkeit besitzen
- Attributdomänen müssen kompatibel sein

Schnittmenge

Herleitung

$$r \cap s = r - (r - s)$$



Beispiel

Gegeben seien die Relationen r und s

r

A	B
11	1
11	2
12	1

s

A	B
11	2
12	3

$r \cap s$

A	B
11	2

Division

Geeignet für Anfragen, die den Ausdruck „für alle“ enthalten (→ Simulation des All-Quantors)

Notation

$$r \div s$$

Division

Geeignet für Anfragen, die den Ausdruck „für alle“ enthalten (→ Simulation des All-Quantors)

Notation

$$r \div s$$

Voraussetzung

Die Attribute in S müssen eine Teilmenge der Attribute in R sein, d. h. $S \subseteq R$

- Sei r, s Relationen über den Schemata R bzw. S, wobei
  - $R(A_1, ..., A_m, B_1, ..., B_n)$
  - $S(B_1, ..., B_n)$
- Das Ergebnis von $r \div s$ ist eine Relation über dem Schema $R - S = (A_1, ..., A_m)$
- Die Ergebnismenge besteht aus den Tupeln von r (über den Attributen $R - S$), die zu allen Tupeln in s passen

Division

Geeignet für Anfragen, die den Ausdruck „für alle“ enthalten (→ Simulation des All-Quantors)

Notation

$$r \div s$$

Voraussetzung

Die Attribute in S müssen eine Teilmenge der Attribute in R sein, d. h. $S \subseteq R$

- Sei r, s Relationen über den Schemata R bzw. S, wobei
  - $R(A_1, ..., A_m, B_1, ..., B_n)$
  - $S(B_1, ..., B_n)$
- Das Ergebnis von $r \div s$ ist eine Relation über dem Schema $R - S = (A_1, ..., A_m)$
- Die Ergebnismenge besteht aus den Tupeln von r (über den Attributen $R - S$), die zu allen Tupeln in s passen

Definition

$$r \div s := \{\, t[A_1, ..., A_m] \mid t \in r \wedge \forall u \in s : (t[A_1, .., A_m] \cup u) \in r \,\}$$

Division

Beispiel

Gegeben seien die Relationen r und s

r	A	B	C	D	E
	11	a	2	a	1
*	11	a	4	a	1
*	11	a	4	b	1
	12	a	4	a	1
	12	a	4	b	3
*	13	a	4	a	1
*	13	a	4	b	1
	13	a	3	b	1

s

D	E
a	1
b	1

$r \div s$

A	B	C
11	a	4
13	a	4

Natürlicher Verbund (Join)

Kombiniert zwei Relationen über identische Werte in gleichnamigen Attributen

Notation

$$r_1 \bowtie r_2$$
(wobei r_1 und r_2 Relationen sind)

Definition

- Relation r mit $m + k$ Attributen: $A_1, ..., A_m, B_1, ..., B_k$
- Relation s mit $n + k$ Attributen: $B_1, ..., B_k, C_1, ..., C_n$
- Annahme: A_i und C_i sind paarweise verschieden $\rightarrow$ nur $B_1, ..., B_k$ sind gemeinsame Attribute
- Stelligkeit von $r \bowtie s$: $m + k + n$

$$r \bowtie s := \Pi_{A_1,...,A_m,r.B_1,...,r.B_k,C_1,...,C_n} \left(\sigma_{r.B_1=s.B_1 \wedge ... \wedge r.B_k=s.B_k}(r \times s)\right)$$

Natürlicher Verbund (Join)

Hinweise

- Ergebnisschema: $R \cup S$
- Wenn es keine gleichnamigen Attribute gibt ($R \cap S = \emptyset$), degeneriert der natürliche Verbund zu einem kartesischen Produkt ($r \times s$)
- Natürlicher Verbund ist ein **Equi-Verbund** bezüglich aller gleichnamigen Attribute

Beispiel

Gegeben seien die Relationen r und s

r

A	B
11	1
12	2
14	4

s

B	C
1	21
2	22
3	23

$r \bowtie s \Longrightarrow$

A	B	C
11	1	21
12	2	22

Natürlicher Verbund (Join)

Beispiel

Gegeben seien die Relationen r und s

r

A	B
11	1
12	2
14	4

s

C	D
1	21
2	22
3	23

$r \bowtie s \Rightarrow$

A	B	C	D
11	1	1	21
11	1	2	22
11	1	3	23
12	2	1	21
12	2	2	22
12	2	3	23
14	4	1	21
14	4	2	22
14	4	3	23

Um unbeabsichtigte kartesische Produkte zu vermeiden, sollte man die Join-Bedingungen immer explizit angeben ⇒ Theta-Join verwenden

Theta-Verbund

Notation

$$r \bowtie_C s$$

C ist eine Bedingung über Attribute in $R \cup S$

Definition

$$r \bowtie_C s = \sigma_C(r \times s)$$

Hinweise

- C wird auch als θ (Theta) bezeichnet
- Wenn $R \cap S \neq \varnothing$ und C diese Attribute referenziert, müssen sie ggf. umbenannt werden
- Das Ergebnisschema ist wie beim natürlichen Verbund
- Wenn C nur Gleichheitsvergleiche enthält, spricht man auch von einem **Equi-Verbund**

Theta-Verbund

Beispiel

Gegeben seien die Relationen r und s

r

A	B	C
1	2	3
4	5	6
7	8	9

s

D	E
3	1
6	2

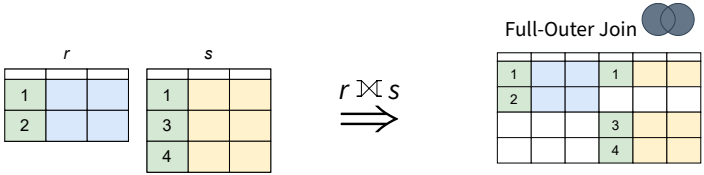
$r \bowtie_{B<D} s \Rightarrow$

A	B	C	D	E
1	2	3	3	1
1	2	3	6	2
4	5	6	6	2

Weitere Verbund-Typen

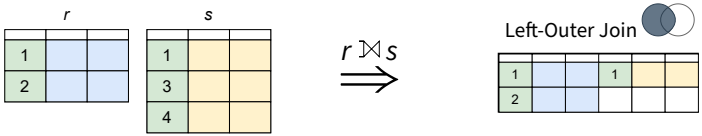
- Theta-Verbund und Natürlicher Verbund werden auch als *Inner Joins* bezeichnet
- Inner Joins beinhalten nur Tupel, die die Verbund-Bedingung erfüllen
- Wenn man auch Tupel ohne passenden Partner einbeziehen möchte, verwendet man **Outer Joins**

Full-Outer Join

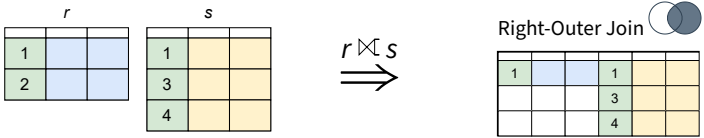


Weitere Verbund-Typen

Left-Outer Join



Right-Outer Join



Gruppierung und Aggregation

- **Gruppierung:** Gruppieren von Tupeln nach gemeinsamen Attributwerten
- **Aggregation:** Anwenden einer Funktion auf jede Gruppe (z. B. `count`, `avg`, `sum`, `min`, `max`)

Notation

$$\gamma_{A_1,\ldots,A_n;F_1,\ldots,F_n}(r)$$

wobei A_i ein Attribut und F_i eine Aggregatfunktion ist

Beispiel

MatrNr	VorlNr	Note
111	042	3.0
112	065	4.0
113	073	2.3
111	065	2.0
112	042	1.0

$\gamma_{MatrNr;avg(Note)}(r)$

MatrNr	avg
111	2.5
112	2.5
113	2.3

$\gamma_{VorlNr;avg(Note)}(r)$

VorlNr	avg
042	2.0
065	3.0
073	2.3

Kombination von Operatoren

- Es ist möglich, Ausdrücke der relationalen Algebra zu verschachteln (wie arithmetische Ausdrücke)
- Ein Basisausdruck in der relationalen Algebra ist entweder:
  - eine Relation aus der Datenbank
  - eine konstante Relation (fester Tupelvorrat, z.B. $\{(1,2),(1,3),(2,3)\}$)

Kombination von Operatoren/2

- Wenn E_1 und E_2 Ausdrücke der relationalen Algebra sind, dann sind auch folgende Ausdrücke gültig:
  - $E_1 \cup E_2$
  - $E_1 - E_2$
  - $E_1 \bowtie E_2$ (Verbund)
  - $\sigma_P(E)$, wobei P ein Prädikat auf Attributen von E ist
  - $\pi_A(E)$, wobei A eine Attributliste aus E ist
  - $\rho_x(E)$, wobei x ein neuer Name für das Ergebnis von E ist

Beispielanfragen

Beispieldaten

Studierende

MNr	Name	Alter	Studienfach
1	Alice	20	Informatik
2	Bob	22	Mathematik
3	Carol	19	Physik
4	Dave	21	Mathematik

Kurse

KNr	Kursname	Leistungspunkte
101	Datenstrukturen	3
102	Analysis	4
103	Physik I	3
104	Algorithmen	3

Einschreibungen

ENr	MNr	KNr	Note
1	1	101	A
2	1	102	B
3	2	102	A
4	2	103	B
5	3	103	A
6	3	104	C
7	4	101	B

Beispielanfragen

Frage: Finde alle Namen von Studierenden, die im Kurs „Datenstrukturen“ eingeschrieben sind.

$$\pi_{Name}\big(\sigma_{Kursname='Datenstrukturen'}(\texttt{KURSE}) \bowtie_{\texttt{KURSE.KNr = EINSCHREIBUNGEN.KNr}} \texttt{EINSCHREIBUNGEN} \bowtie_{\texttt{STUDIERENDE.MNr = MNr}} \texttt{STUDIERENDE}\big)$$

Ergebnis

Name
Alice
Dave

Beispielanfragen

Frage: Finde Namen der Studierenden, die in einem Kurs die Note „A“ erhalten haben.

$$\pi_{Name}(\sigma_{Note='A'}(\texttt{EINSCHREIBUNGEN}) \bowtie_{\texttt{STUDIERENDE.MNr = MNr}} \texttt{STUDIERENDE})$$

Ergebnis

Name
Alice
Bob
Carol

Beispielanfragen

Frage: Liste alle Studierenden auf, die in mehr als einem Kurs eingeschrieben sind.

$$\pi_{Name}\left(\sigma_{\texttt{AnzKurse}>1}\left(\gamma_{\texttt{MNr; count(KNr)}\rightarrow\texttt{AnzKurse}}(\texttt{EINSCHREIBUNGEN})\right) \bowtie_{\texttt{STUDIERENDE.MNr = MNr}} \texttt{STUDIERENDE}\right)$$

Ergebnis

Name
Alice
Bob
Carol

Beispielanfragen

Frage: Finde Kurse, in denen kein*e Studierende*r die Note „A“ erhalten hat.

$$\pi_{\mathit{Kursname}}\left((\pi_{\mathit{KursNr}}(\texttt{KURSE}) - \pi_{\mathit{KursNr}}(\sigma_{\texttt{Note}='A'}(\texttt{EINSCHREIBUNGEN}))) \bowtie_{\texttt{KURSE.KNr}=\texttt{KNr}} \texttt{KURSE}\right)$$

Ergebnis

Kursname
Algorithmen

Weitere Beispielanfragen

Finde die Namen der Studierenden, die in keinem Kurs eingeschrieben sind.

Finde die Namen der Studierenden, die sowohl in „Datenstrukturen“ als auch in „Analysis“ eingeschrieben sind.

Berechne das durchschnittliche Alter der Studierenden, die in jedem Kurs eingeschrieben sind.

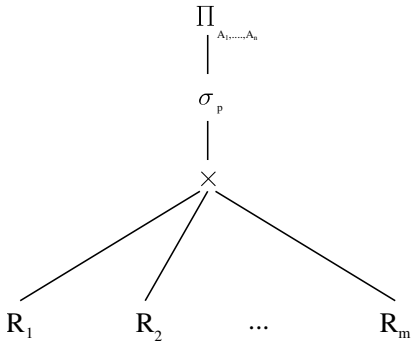
Finde die Namen der Studierenden, die „Mathematik“ studieren und im Kurs „Physik I“ eingeschrieben sind.

Finde alle Studierenden, die insgesamt mehr als 3 Leistungspunkte belegen.

Grafische Darstellung relationaler Algebra-Ausdrücke

- Relationale Algebra-Ausdrücke können grafisch in einer baumartigen Struktur dargestellt werden
- Knoten im Baum repräsentieren Operationen (Ausdrücke)
- Kanten im Baum stellen den Datenfluss zwischen Operationen (Relationen) dar

Beispiel



Zusammenfassung

- Relationale Algebra als theoretische Grundlage von Anfragesprachen (z. B. SQL)
- Basierend auf dem relationalen Modell und darauf abgeschlossen (Eingabe und Ausgabe jeder Operation sind Relationen)
- Die Menge an Basisoperationen ist minimal (d. h. 6 Stück)
- Komplexere Operationen (z. B. Joins) können daraus zusammengesetzt werden